

SPM Modular Project: Modulo3

SPM course a.y. 25/26

Partitioned Hash Join with Duplicates

Module 3: Parallel Implementation using OpenMP

In the previous module, you implemented a complete end-to-end version of the partitioned hash join and designed a parallel implementation using C++ concurrency mechanisms. In this module, you will study the same algorithm using OpenMP. The goal is not simply to rewrite the code with OpenMP pragmas, but to understand how different OpenMP constructs behave under different workload conditions, and how they compare with the C++ concurrent implementation developed in Module 2.

The overall algorithm remains unchanged. As in the previous module, both input relations are partitioned according to the mapping function. Records assigned to the same partition are grouped into contiguous memory regions, and the join is then performed independently on each partition. The final result is obtained by accumulating the contributions of all partitions.

Sequential Reference Implementation

The OpenMP implementation must be evaluated against the same sequential baseline used in Module 2. If you improved the provided sequential implementation in Module 2, you must use that same improved sequential version as the baseline in this module as well. The purpose is to ensure a fair comparison between execution models, not to compare different algorithms.

OpenMP Implementation

You are required to implement an OpenMP version of the algorithm. The implementation must preserve the same overall algorithmic structure used in the sequential reference code and in your C++ threads version. It must produce exactly the same output, using the same mapping function and the same input data.

You must provide:

- one OpenMP implementation based on loop-level parallelism
- one OpenMP implementation based on tasks

The purpose of this comparison is to understand when a simple loop-based strategy is sufficient and when a task-based strategy becomes beneficial. This comparison should be carried out under both uniform and skewed workloads.

Workload and Input Data

You must consider at least two input scenarios:

- Uniform dataset
 - use the same input generation scheme adopted in Module 2
- Skewed dataset.
 - Design and generate an input dataset in which the workload is not evenly distributed across partitions. For example, a large fraction of the keys may be chosen so that they map to a small subset of partitions.

The goal is to create a case in which different partitions have significantly different workloads.

Correctness

All implementations must produce the same output, namely:

- the total number of matches (join_count)
- the checksum values used for verification

As in the previous modules, you must implement a validation strategy. For small inputs, stronger checks can be performed, for example against a simpler reference implementation. For larger inputs, correctness is verified by comparing the final aggregated output values (join_count, checksum1, and checksum2) across all implementations. The verification mechanism should be kept separate from the measured region.

Performance Evaluation

You must evaluate the performance of your OpenMP implementations with respect to the sequential baseline and to your C++ threads implementation. Consider speedup, strong and weak scalability. You are encouraged to provide a breakdown of execution time across the main phases of the algorithm. The report must include a discussion of the impact of workload skewness on performance. All experiments must be performed on one of the nodes of the spmcluster.

Notes

You are expected to make reasonable engineering choices and justify them. There is no single correct OpenMP strategy, provided that correctness is preserved, the overall algorithmic structure remains recognizable, and the adopted choices are justified through analysis and experiments.

Deliverables

Send the teacher an email with subject “SPM Modulo3” which has attached a zip file with the name Modulo3_NameSurname.zip **by 10 May** (soft deadline). The zip file must contain:

- Source code and build instructions (Makefile or explicit compile commands).
- README containing instructions on how to run the tests and perform correctness checks.
- A short PDF report (max 5 pages) that includes:
 - The OpenMP strategies adopted
 - How the skewed dataset was generated
 - Performance results obtained on both uniform and skewed workloads on one of the nodes of the spmcluster.
 - Brief discussion on the results obtained and comparison with C++ threaded version from Module 2